International Journal of Software Engineering
and Knowledge Engineering
Vol. 30, No. 6 (2020) 835–857
© World Scientific Publishing Company
DOI: 10.1142/S0218194020400094

World Scientific
www.worldscientific.com

Int. J. Soft. Eng. Knowl. Eng. 2020.30:835-857. Downloaded from www.worldscientific.com
by UNIVERSITY OF NEW ENGLAND on 08/10/20. Re-use and distribution is strictly not permitted, except for Open Access articles.

Semantic Restful Service Composition Using Task Specification

Shang-Pin Ma*, Hsuan-Ju Lin† and Ming-Jen Hsu‡

Department of Computer Science and Engineering
National Taiwan Ocean University
Keelung City 20224, Taiwan, R. O. China
**albert@ntou.edu.tw*
†*mis101bird@gmail.com*
‡*surprised128@gmail.com*

Received 18 December 2019
Revised 3 March 2020
Accepted 7 April 2020

Existing Web API search engines allow only category-based browsing and keyword- or tag-based searches for RESTful services. In other words, they do not enable the discovery or composition of real-world RESTful services by application developers. This paper outlines a novel scheme, called Transformation–Annotation–Discovery (TAD), which transforms OpenAPI (Swagger) documents related to RESTful services into a graph structure and then automatically annotates the semantic concepts on graph nodes using Latent Dirichlet Allocation (LDA) and WordNet. TAD can then be used for service composition based on the user requirements specified in two modules: a service discovery chain and logical-operation-based composition. The service discovery chain uses the Hungarian algorithm to assess service interface compatibility in order to facilitate the retrieval of services capable of bridging the gap between specified user requirements and the discovered services. The logical-operation-based composition module identifies services that semantically fit the user requirements, based on the structure of the service flow. Those candidate services are then sent to service discovery chains to enable the simultaneous search for potential composition solutions. System prototype and experiment results demonstrate the feasibility and efficacy of the proposed scheme.

Keywords: RESTful service composition; RESTful service discovery; semantic annotation; graph database.

1. Introduction

Information systems and software applications using Web services (also called Web APIs) are growing in popularity due to their flexibility and low cost. Numerous information technology (IT) companies, such as Google, Facebook, Netflix, eBay, LinkedIn, Foursquare, and Instagram, have published Web services to enable access

* Corresponding author.

Int. J. Soft. Eng. Knowl. Eng. 2020.30:835-857. Downloaded from www.worldscientific.com
by UNIVERSITY OF NEW ENGLAND on 08/10/20. Re-use and distribution is strictly not permitted, except for Open Access articles.

by external users (consumers). At present, a large number of existing Web services employ the REST (REpresentational State Transfer) architecture [1]. There are more than 10,000 registered RESTful Web services on ProgrammableWeb [2], and this number is increasing rapidly. This has led to the development of numerous applications based on public Web APIs via service discovery and composition. In service composition, matching must be performed for functionality as well as the inputs and outputs of the various services. Unfortunately, existing Web API search engines, such as ProgrammableWeb, APIs.io [3], Mashape [4], and APIs.guru [5], allow only category-based browsing and keyword- or tag-based searches. This greatly hinders the discovery and composition of suitable RESTful services. The cost (in terms of time and expense) of searching through the enormous number of available RESTful Web services has necessitated the development of an automated discovery and composition scheme. A number of RESTful Web service composition methods have already been developed [6]. Considerable effort has gone into the formulation of service-specific Web API description specifications [7–9]; however, the adoption of these protocols by service providers as well as service requesters has met with some resistance. Other service composition methods rely on fixed workflows to perform service composition [10–15] and seek optimal service combinations based on indicators, such as QoS (Quality of Service) or reputation. The assumption of this research track is that the involving abstract component services and their sequence in the composition are fixed, but the binding concrete services can be discovered and selected at runtime [16]. Unfortunately, this approach cannot dynamically alter the workflow or the number of component services as need dictates; it makes the approach much less practical.

In this study, we sought to overcome the above issues by automating the discovery and composition of RESTful Web services. The proposed scheme, called Transformation–Annotation–Discovery (TAD), is shown in Fig. 1. TAD is based on

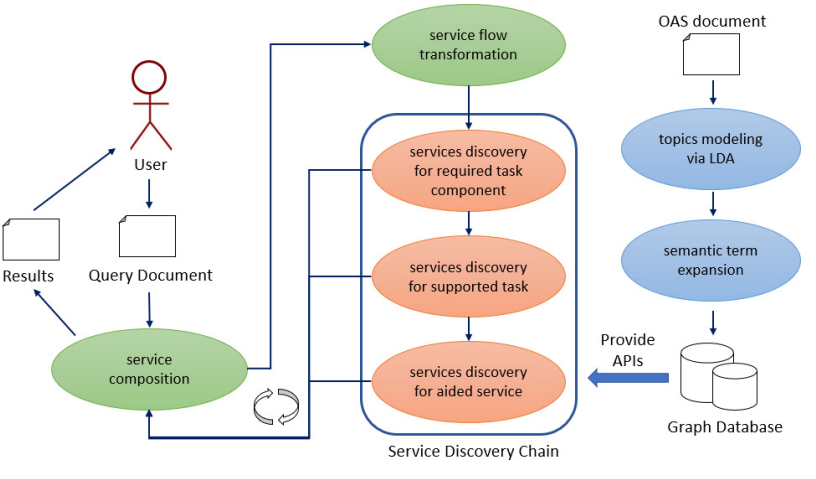


Fig. 1. Overview of the proposed TAD scheme.

Int. J. Soft. Eng. Knowl. Eng. 2020.30:835-857. Downloaded from www.worldscientific.com
by UNIVERSITY OF NEW ENGLAND on 08/10/20. Re-use and distribution is strictly not permitted, except for Open Access articles.

the widely-adopted OpenAPI Specification (OAS, also called Swagger) [17] to perform service discovery and service composition. TAD transforms OAS documents into graph nodes within a graph database (DB) and then performs semantic annotation on these nodes using Latent Dirichlet Allocation (LDA) [18] and WordNet-based [19] term expansion. The proposed transformation and annotation processes make it easy to discover services listed in the graph DB. The proposed service discovery chain (SDC) checks service interface compatibility to enable the retrieval of supported services to bridge the gap between requirements stipulated by the user and a set of candidate services discovered using the Hungarian algorithm. The logical-operation-based composition (LOBC) module identifies services that semantically fit user requirements, based on the structure of the service flow. Those candidate services are then sent to service discovery chains to enable the simultaneous search for potential composition solutions that meet user requirements. The proposed scheme allows the developers to simply submit a QD (Query Document) describing the input parameters, output parameters, and a sequence of tasks (SOT), whereupon TAD returns a list of candidate service compositions that fit the user requirements. Note that TAD is able to search for real-world RESTful services and produce feasible service compositions as long as the OAS documents are available. TAD is more flexible than the existing fixed-workflow methods, thanks to the inclusion of multiple compatible services.

The remainder of this paper is organized as follows. Section 2 outlines the background work related to this study and presents a review of related researches. Section 3 describes the details of the proposed TAD approach. Sections 4 and 5 present the system prototype and the experiment results supporting the proposed approach. Conclusions and future work are presented in Sec. 6.

2. Related Work

In the past, the service composition problem was usually mapped as a QoS optimization problem for a fixed workflow. Zeng *et al.* [10, 11] presented a QoS model for the assessment of nonfunctional qualities and proposed a method for identifying the optimal composition using mixed integer programming (MIP). Aggarwal *et al.* [12] proposed a framework for QoS constraint-driven service composition as well as a tool that integrates Web services via integer programming. Yu and Lin [13] designed service selection algorithms for composite services by transforming the selection problem into a multiple-choice knapsack problem (MCKP). Qi *et al.* [14] proposed a heuristic service composition method called LOEM (Local Optimization and Enumeration Method) to find near-to-optimal compositions via the MIP. Alrifai *et al.* [15, 20] presented a hybrid solution that combines global optimization with local selection techniques to address the performance of normal global optimization for service composition. The above methods are not easily applied in real-world cases of composing public services, such as Google Search APIs, Google Map APIs, or Yahoo

Int. J. Soft. Eng. Knowl. Eng. 2020.30:835-857. Downloaded from www.worldscientific.com
by UNIVERSITY OF NEW ENGLAND on 08/10/20. Re-use and distribution is strictly not permitted, except for Open Access articles.

Weather APIs, due to three kinds of difficulties: (1) It is hard to dynamically alter the workflow or the number of component services as need dictates; (2) it is not always feasible to find multiple interface-compliant services for a predetermined workflow since APIs developed by different organizations always provide different service interfaces; and (3) it is also impractical to obtain sufficient historical QoS data for all candidate services from different organizations.

Graph theory has also been applied to the problem of service composition. Grigori *et al.* [21] suggested that service discovery relies on the inputs and outputs of services as well as the conversation protocol specified by BPEL (Business Process Execution Language) or WSCL (Web Services Conversation Language). They reduced the problem of behavioral matching to a graph matching problem and proposed an algorithm to calculate the cost of graph transformation from a candidate service conversation model to an input model. Cuzzocrea and Fisichella [22] developed an algorithm for the discovery of composite OWL-S processes in which two graph representations are matched, and then the degree to which they match is calculated by combining the similarities among atomic services and the similarities in the control flow among them. Note that all of the above methods rely on a predetermined composition model in BPEL, WSCL, or OWL-S to perform matching, despite the fact that few composition specifications have been published.

The third approach is the RESTful service composition. The JOpera visual composition language [8] was the first attempt to provide a composition environment that supports the features of dynamic late binding, dynamic typing, content-type negotiation, state inspection, and compliance with the uniform interface principle for RESTful services. This iterative approach to composition uses feedback to improve composition models. Alarcon *et al.* presented a hypermedia-driven RESTful service method [9] based on the ad-hoc language ReLL (Resource Linking Language) and Petri nets. ReLL is a description language that satisfies the HATEOAS (Hypermedia As The Engine Of Application State) principle, whereas Petri nets allow the modeling and early evaluation of compositions. Alarcon *et al.* [7] also proposed a service description model that allows the generation of graphs capable of capturing state transitions in an activity layer as well as resources, transitions, and response semantics in a semantic layer. This method considers the features of the REST architecture, while leveraging the performance of graph databases. The service description model allows users to traverse the graph using graph query language in order to facilitate the manual discovery and composition of services. Ed-douibi *et al.* [23] presented a data-driven approach to RESTful service composition, called API-Composer, in which OpenAPI definitions are parsed to create a global data model representing the union of all data for APIs. Users are able to perform queries using the proposed OData query language to locate multiple APIs used in composite services. Note that the methods mentioned above permit RESTful service composition; however, they do not consider interface compatibility between services and do not support fully automated service composition.

Int. J. Soft. Eng. Knowl. Eng. 2020.30:835-857. Downloaded from www.worldscientific.com
by UNIVERSITY OF NEW ENGLAND on 08/10/20. Re-use and distribution is strictly not permitted, except for Open Access articles.

3. The Proposed TAD Approach

In this section, we outline the proposed system, Transformation–Annotation–Discovery, which is shown in Fig. 1. Our system includes three main mechanisms: (1) newly-devised specifications for user query documents; (2) graph mapping strategies for the transformation of OAS into graph nodes in the Neo4J database [24] and semantic annotation; and (3) a graph-DB-based service composition algorithm. Note that the service composition algorithm includes three mechanisms: (a) service similarity calculation methods; (b) a service discovery chain that includes four handlers: Pre-Processing Handler (PPH), Required Step Handler (RSH), Supported Step Handler (SSH), and Aided Step Handler (ASH); and (c) an LOBC algorithm to handle service composition tasks.

3.1. *TAD query document*

To allow the users to easily provide their requirements for service composition, we devised the specification of QD, which is divided into three parts: input parameters, output parameters, and the sequence of tasks. The term *input parameter* refers to the information provided by the user to describe the required service. In QD, a group of semantic concepts can be provided to describe an input parameter. Users are encouraged to provide input parameters with multiple semantic concepts to increase the likelihood of identifying matching services. Taking a translation API as an example, if a user wants to provide input parameters for the task of converting audio to text in a specific language, he/she could provide a group of concepts, such as [["*audio*", "*stream*"], ["*locale*", "*language*", "*target*"]], as input parameters.

The term output parameter refers to information describing what the user expects from the returned composite service. As with the input parameters, we can again use a group of semantic concepts to represent a single output parameter. Take an image API as an example, in which the user wishes to obtain the title and URL of an image. He/she could enter a group of concepts, such as[["*image*", "*text*", "*title*"]or ["*image*", "*url*", "*link*"]], as output parameters.

The sequence of tasks refers to the ordered list of service tasks expected by the user. There are two types of tasks: RT (required task) and ST (supported task). RTs are used to describe the semantic concepts of the required services. For example, a simple SOT, ["*audio to text*", "*translate text*", "*text to speech*"], may be specified to represent three sequential user-required tasks. After semantically analyzing a target RT, it should be possible to find any number of candidate services. ST is used to forecast supported services that are likely to cooperate with the target RT in fulfilling the requirements specified by the user. The syntax of ST in QD includes a * in front of the RT. The inclusion of supported services is meant to provide the arguments required for a candidate service. In other words, if the input parameters stipulated by the user do not cover the inputs of a candidate service, then TAD will attempt to find supported service capable of making the available service invocable. Note that QD also supports logical operators. According to [25], the service composition processes

Int. J. Soft. Eng. Knowl. Eng. 2020.30:835-857. Downloaded from www.worldscientific.com
by UNIVERSITY OF NEW ENGLAND on 08/10/20. Re-use and distribution is strictly not permitted, except for Open Access articles.

can be divided into four categories: sequence, parallel, branch, and loop. The sequence is naturally supported. Parallel refers to the simultaneous execution of multiple paths. We use a logical operator [AND] to express this flow pattern. Branch refers to conditional paths. We use a logical operator [OR] to represent this flow pattern. The loop process is seldom encountered in actual service composition due to high computational overhead. Our approach does not currently support the loop pattern.

A complex example of SOT is presented in Fig. 2. In this example, a developer plans to build a mobile App to perform simple language translations. When a user sends a voice message in Chinese to this App, it translates the sentences from Chinese to English and verbally reads out the translated sentences in English. We need to realize that this service requires three types of RESTful service: speech to text, text translation, and text to speech. Specifying inputs, outputs, and SOT in QD makes it possible for TAD to return a number of candidate service compositions to the user. An example of QD in the JSON format is shown in Fig. 3. The QD is specified for the application of translation from a voice in a language to a voice in another language. In the example, two input parameters and an output parameter were specified. These parameters consist of 2–3 concepts. The SOT includes three required tasks ("audio to text", "translate text", and "text to speech") and a supported task ("detect language").

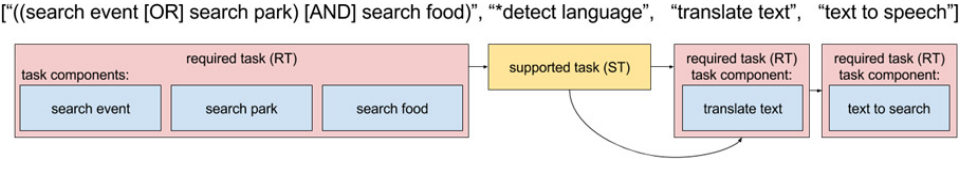


Fig. 2. Example of a complex SOT.

```
{
  "inputs": [
    [ "audio", "stream" ],
    [ "target", "language", "locale" ]
  ],
  "outputs": [
    [ "audio", "stream" ]
  ],
  "sot": [ "audio to text", "*detect language", "translate text", "text to speech" ]
}
```

Fig. 3. Example of a QD.

3.2. *Relational mapping from OAS to graph*

To facilitate the matching of a user requirement (i.e. QD) and OAS documents, we decided to apply the graph database due to the fact that the graph database is very suitable to represent connected resources and their linkages, e.g. the linkages between semantic concepts in ontologies and the ingredients in service descriptions.

Int. J. Soft. Eng. Knowl. Eng. 2020.30:835-857. Downloaded from www.worldscientific.com
by UNIVERSITY OF NEW ENGLAND on 08/10/20. Re-use and distribution is strictly not permitted, except for Open Access articles.

Besides, via the built-in query capability, the retrieval of connected resources according to the QD is relatively simple. Accordingly, in TAD, an OAS is transformed into a graph structure, which is divided into two layers: a concrete service ingredient layer (CSIL) and a semantic concept layer (SCL), and stored in the graph database, Neo4J. Each node in either CSIL or SCL has a specific label (i.e. node name) and properties. CSIL keeps all the specific information on the OAS document, such as title, provider, path, and so on. SCL annotates the semantic concepts on each ingredient in CSIL. SCL is responsible for managing the semantic concepts for each ingredient in CSIL. The interconnection of nodes in these two layers is based on a relationship referred to as "reference". For example, the node named Resource connects to the node named ResourceConcept (RC), the node named Operation links to the node named OperationConcept (OC), and the node named Parameter/Response refers to the node named ParameterConcept. The semantic layer enables users to employ semantic concepts to find the API they require and then grab the concrete API information from nodes in CSIL by exploiting the reference relationships. The graph construction process is motivated by the method described in [7], whereas the details of the graph construction can be found in [26], our initial research result for TAD. Figure 4 presents an example of a graph constructed for the speech-to-text API provided by IBM. The nodes in SCL are shown in red, and the nodes in CSIL are shown in blue. There are multiple "reference" links between nodes in two layers. For example, the Parameter node ["voice", "stream"] in CSIL was linked to the ParameterResource node ["audiotext"] in SCL.

One of the important steps of the annotation process is retrieving keyword terms by LDA for the concept nodes, due to the fact that LDA is a commonly-used technique to statistically discover topics for documents. To reduce computation complexity, we keep only three retrieved topics for each concept node, and pick two most representative terms for each topic. Next, TAD also performs WordNet-based term expansion for each retrieved topic term (please also refer to [26] to check details). WordNet is a large lexical database of the English language, in which nouns, verbs, adjectives, and adverbs are grouped into sets of synsets, each expressing a distinct concept. By using WordNet, TAD fetches the synset, i.e. synonyms, hypernyms, and hyponyms, for each term to be a candidate expanded term, and filters out candidates whose similarity to the original term is lower than a given threshold by Li et al.'s algorithm [27].

When the annotation process is finished, TAD obtains a list of original concept terms and a list of expanded terms. The terms are stored in the SCL and linked to the original nodes in the CSIL. Each node in CSIL refers to at least one node in SCL. This ensures that users are able to find the concrete service they require simply by querying nodes in the semantic concept layer. Note that the nodes with the same semantic concept in CSIL refer to the same node in SCL. This feature is helpful in finding RESTful services using semantic concepts. We relied heavily on this idea in the development of our service composition scheme.

Int. J. Soft. Eng. Knowl. Eng. 2020.30:835-857. Downloaded from www.worldscientific.com
by UNIVERSITY OF NEW ENGLAND on 08/10/20. Re-use and distribution is strictly not permitted, except for Open Access articles.

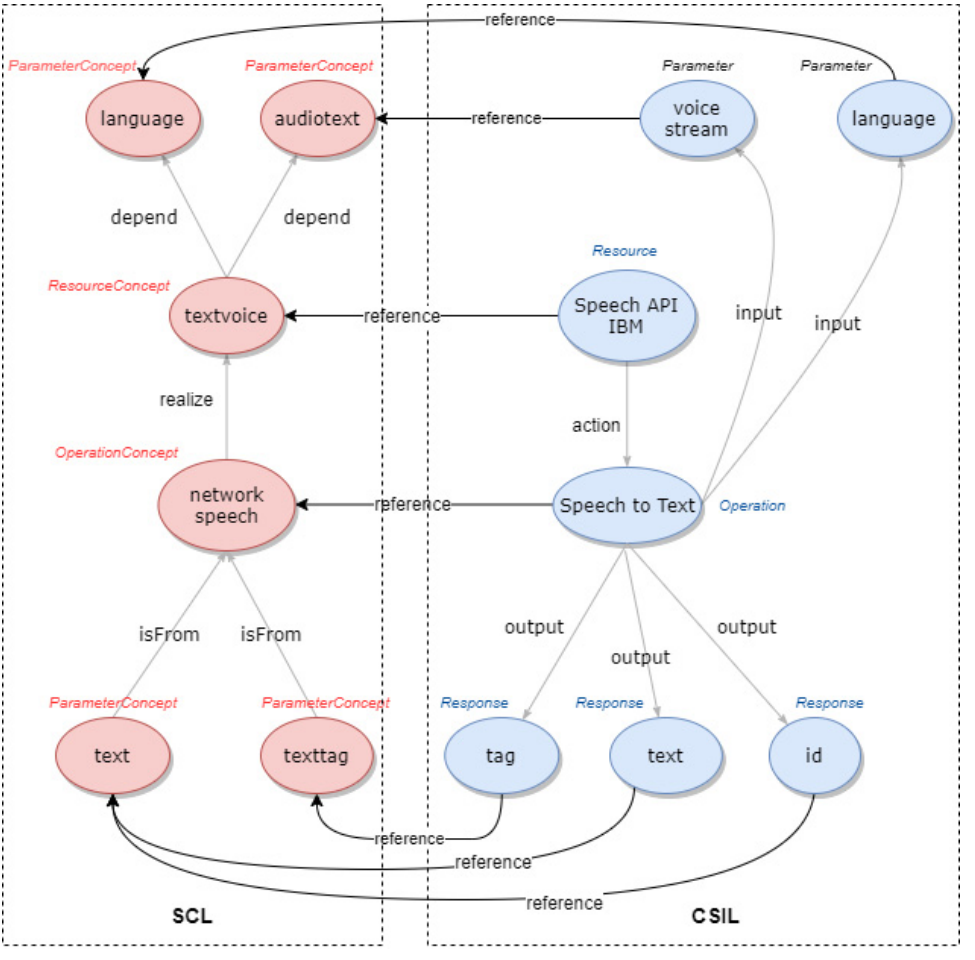


Fig. 4. Graph structure of the speech-to-text API.

3.3. *Graph-DB-based service composition algorithm*

In this sub-section, we first introduce the methods used in TAD for the calculation of similarity. We then take a brief overview of the TAD composition algorithm, discuss the interactions among modules, and describe the concepts of LOBC and SDC.

3.3.1. *Service similarity calculation methods*

During the service composition, a lot of matching jobs should be performed, including the matchings between the input/output parameters of a service and the parameters specified in QD, between a service operation and a task in QD, and between a service composition and the whole SOT. To facilitate the matching, computing similarities between a source and a target at deferent levels are necessary.

Int. J. Soft. Eng. Knowl. Eng. 2020.30:835-857. Downloaded from www.worldscientific.com
by UNIVERSITY OF NEW ENGLAND on 08/10/20. Re-use and distribution is strictly not permitted, except for Open Access articles.

In the following, we outline some important definitions and describe the methods used for the calculation of similarity before examining the composition algorithm.

(1) Input Pool (IP): The container of available input parameters in the composite process.
(2) Output Pool (OP): The container of output parameters required by the user.
(3) Concept Score (CS): The score between the semantic concepts of a task and RC and OC of candidate service. CS calculates similarity based on the VSM (Vector Space Model). For the RC or OC of a service, the original concept list is transformed into a vector V_{oc} and the expanded WordNet concept list is expanded into a vector V_{wc}. For any task in QD, the concepts of the task are transformed into a vector V_t. We then calculate the similarity score between the vector of RC/OC and the vector of a task related to a candidate service using VSM, as follows:

$$\text{sim}(\text{C}, T) = \frac{V_{\text{oc}} \cdot V_t + V_{\text{wc}} \cdot V_t}{\|V_t\| \cdot \|V_{\text{oc}}\|}, \tag{1}$$

where C is RC or OC, RC is the ResourceConcept (a node of the SCL representing the semantic concepts of the Resource node of a candidate service), OC is the OperationConcept (a node of the SCL representing the semantic concepts of the Operation node of a candidate service), and T is the task. After performing these calculations, we can obtain a CS by averaging $\text{sim}(S.\text{rc}, T)$ and $\text{sim}(S.\text{oc}, \text{TC})$, as follows:

$$\text{CS}(S, T) = \frac{\text{sim}(S.\text{rc}, T) + \text{sim}(\text{S.oc}, T)}{2}, \tag{2}$$

where S is the candidate service, S.rc is the RC of S, and S.oc is the OC of S. In the LOBC module, if the CS of a candidate service is lower than the given threshold, then the service is discarded.
(4) Parameter Concept Score (PCS): PCS indicates the similarity between two ParameterConcept nodes, PC_t (matching target) and PC_s (matching source), based on VSM. ParameterConcept is the node in SCL which represents the semantic concepts of the Parameter/Response node. The formal representation of vectors of PC_t and PC_s are $\text{PC}_t = \{V_{\text{toc}}\}$, including only the vector of the original concepts, and $\text{PC}_s = \{V_{\text{soc}}, V_{\text{swc}}\}$, including vectors of the original concepts and expanded WordNet concepts. The calculation method is as follows:

$$\text{PCS}(\text{PC}_t, \text{PC}_s) = \frac{V_{\text{toc}} \cdot V_{\text{soc}} + V_{\text{toc}} \cdot V_{\text{swc}}}{\|V_{\text{oc1}}\| \cdot \|V_{\text{oc2}}\|}. \tag{3}$$

(5) Hungarian Mapping Score (HMS)/Adjusted Hungarian Mapping Score (AHMS): HMS/AHMS indicates the degree of interface compatibility between two types of service: $\text{HMS}_{\text{input}}(S\text{IP})$ and $\text{HMS}_{\text{output}}(C, \text{OP})$. The concept of

Int. J. Soft. Eng. Knowl. Eng. 2020.30:835-857. Downloaded from www.worldscientific.com
by UNIVERSITY OF NEW ENGLAND on 08/10/20. Re-use and distribution is strictly not permitted, except for Open Access articles.

interface compatibility is based on the notion of IOC (Input–Output Covering) proposed in our previous work [28]:

- $\mathrm{HMS}_{\mathrm{input}}(S, \mathrm{IP})$: TAD calculates the input matching level between the input parameters of a candidate service and input parameters in IP. S refers to a candidate service, and IP refers to an Input Pool. This type of scoring function is used in the service discovery chain.
- $\mathrm{HMS}_{\mathrm{output}}(C, \mathrm{OP})$: TAD calculates the degree to which the output parameters of a composite service match the expected output parameters in OP. Crefers to a candidate service composition, whereas OP refers to the Output Pool. This type of scoring function is used in the closure phase of service composition.

The calculation steps are as follows:

- For the input part, TAD calculates PCS between each parameter of IP and each input parameter of the candidate service. For the output part, TAD calculates PCS between each parameter of OP and each output parameter of the candidate service.
- TAD then applies the Hungarian algorithm to find the best combination using PCS in order to rate matching pairs. For inputs, TAD divides the total rating of the best combination by the count of the service input parameters to obtain $\mathrm{HMS}_{\mathrm{input}}(S, \mathrm{IP})$. For outputs, TAD divides the total rating of the best combination by the count of OP parameters to obtain $\mathrm{HMS}_{\mathrm{output}}(C, \mathrm{OP})$.
  AHMS is a variant of HMS used to assist in evaluating candidate services that do not fully satisfy the user's needs. In calculating AHMS, TAD applies the Hungarian algorithm between the parameters of a candidate service and the Input Pool. It then discards inappropriate parameters based on a given threshold and calculates the HMS again between the qualified parameters and IP. In other words, AHMS is the result of HMS without the skipped parameters.

(6) Result Score (RS): RS refers to the final score used to sort the candidate composition results and discard inappropriate compositions, the method of which is shown in Eq. (4). RS takes only candidate services of RT into consideration, because they are the services that users actually require:

$$\mathrm{RS}(C, \mathrm{SOT}) = \frac{\mathrm{HMS}_{\mathrm{output}}(C, \mathrm{SOT.op}) + \frac{\sum_{i=1}^{n} \frac{\mathrm{CS}(S_i.\mathrm{RT}_i) + \mathrm{HMS}_{\mathrm{input}}(S_i.\mathrm{RT}_i)}{2}}{n}}{2}, \tag{4}$$

where C is the candidate composition, SOT is the sequence of tasks given by the user, SOT.op is the Output Pool of SOT, S_i is the ith service of RT in the candidate composition, RT_i is the ith RT in SOT, and n is the number of RTs in SOT.

In the proposed service composition algorithm, the Hungarian algorithm is used strongly to find optimal matching between two sets of parameters, either (1) from the input parameters in IP to the input parameters of a candidate service; or (2) from the output parameters of the current composition to the output parameters in OP.

Int. J. Soft. Eng. Knowl. Eng. 2020.30:835-857. Downloaded from www.worldscientific.com
by UNIVERSITY OF NEW ENGLAND on 08/10/20. Re-use and distribution is strictly not permitted, except for Open Access articles.

Table 1. Optimal combination of mappings using the Hungarian algorithm.

Source \ Target	["name"]	["city", "address"]	["coordinate"]	["company"]
["name"]	**1.00**	0.23	0.24	0.42
["latitude", "longitude"]	0.25	0.55	**0.86**	0.22
["prefecture", "region"]	0.18	**0.73**	0.52	0.32

For example, we take the parameters in IP as ["name"], ["city", "address"], ["coordinate"], and ["company"], and the input parameters of a candidate service as ["name"], ["latitude", "longitude"], and ["prefecture", "region"]. Assume that the PCS scores for all parameter pairs are calculated, as shown in Table 1. The optimal combination of mappings (see numbers highlighted in bold in Table 1) can be determined using the Hungarian algorithm: {(["name"], ["name"]), (["city", "address"], ["prefecture", "region"]), (["coordinate"], ["latitude", "longitude"])}. Both HMS and AHMS are calculated based on the determined mapping of parameters.

3.3.2. *Service composition algorithm*

TAD includes two primary modules: LOBC and SDC. The LOBC module finds services that semantically fit the users' specifications based on the annotated graph DB, and then sends candidate services to SDCs to simultaneously seek multiple composition solutions for the SOT provided by the user. SDC is tasked with determining whether the parameters of IP are capable or nearly capable of invoking a candidate service and retrieving supported or aided services to bridge the gap between the user requirements and the services that have been discovered. During the composition phase, the discovered services are saved in a DSB (Discovered Services Bean).

Figure 5 presents an illustrative example of the proposed logical-operation-based service composition. In this example, LOBC first finds two candidate services (A and B) for the first RT based on CS, and encloses them in DSBs. Second, LOBC allocates two service discovery chains to check interface compatibility between

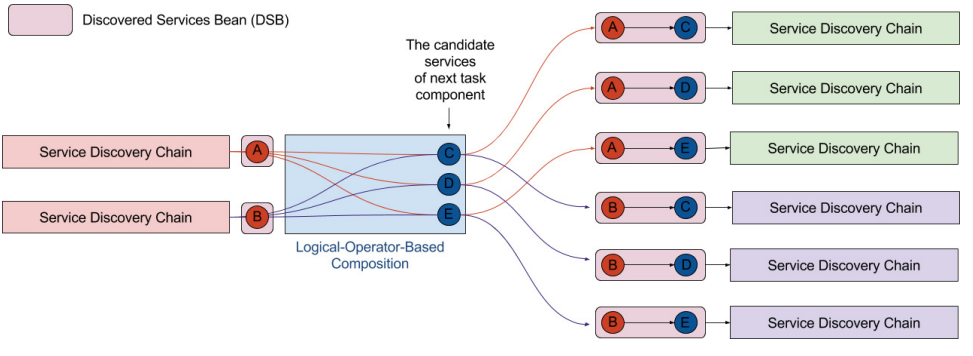


Fig. 5. Cooperation between LOBC and SDC.

Int. J. Soft. Eng. Knowl. Eng. 2020.30:835-857. Downloaded from www.worldscientific.com
by UNIVERSITY OF NEW ENGLAND on 08/10/20. Re-use and distribution is strictly not permitted, except for Open Access articles.

service A/service B and IP based on the AHMS. In this situation, we assume that both services (A and B) survive. Third, LOBC finds three qualified services (C, D, and E) for the next RT and encloses the current composition in DSBs. LOBC then allocates six SDCs to check interface compatibility between the current compositions and IP. The composition process is executed iteratively until all component services for the given SOT are retrieved.

The Service Flow Transformation (SFT) module is tasked with processing the flow logic within LOBC. The following task component (TC) is arranged according to the current input DSB, and the allocation of threads is based on different logical operators. SFT pre-processing includes the following steps: (1) transforming RT into an ordered one-dimensional array with logical operators. For example, "((A [AND] B) [OR] C)" is converted into: ["A [AND] B", "[OR] C"]; and (2) allocating threads to TCs based on related logical operators. For example, in "A [AND] B", A and B are both allocated one thread corresponding to their operator "AND". The rules of thread allocation include the following:

(1) "AND" operator: If the SOT has the logical operator AND, then SFT allocates two threads to the two TCs on either side of the AND. The two threads, respectively, obtain candidate services (possibly with supported services, which will be discussed later) using the procedures described above. SFT then produces multiple DSBs presenting all possible combinations of services for the two TCs. For example, to handle the SOT "(video [AND] music)", SFT allocates two threads to the service discovery involving the tasks "video" and "music". If two candidate services are identified for "video" and three candidate services are identified for "music", then SFT produces six DSBs with two services.
(2) "OR" operator: If the SOT has the logical operator OR, then SFT also allocates two threads to the two TCs on either side of the OR. SFT then produces multiple DSBs presenting the retrieved candidate services uniting the two TCs. For example, when dealing with the SOT "(video [OR] music)", the identification of two candidate services for "video" and three candidate services for "music" will result in five DSBs with only one service.

Figure 6 illustrates the application of the above rules when using various logical operators to deal with RTs. We borrowed the notation of the commonly-used AND–OR tree [29] in the field of artificial intelligence to illustrate the operational concepts of our composition process. In the beginning, after converting "((Video [OR] Music) [AND] (Parking Lot))" into an ordered array ["Video [OR] Music", "[AND] (Parking Lot)"], SFT determines that there is an AND operator in the flow following "Video [OR] Music". Accordingly, SFT allocates two threads for the Video (RT) and the Music (RT) to find candidate services in two separate searches (steps 1-1 and 1-2 in Fig. 6). When the threads of the Video and Music RTs are completed, we obtain three qualified candidate services for the Video task and two qualified candidate services for the Music task. SFT then unites the candidate services of the Video and

Int. J. Soft. Eng. Knowl. Eng. 2020.30:835-857. Downloaded from www.worldscientific.com
by UNIVERSITY OF NEW ENGLAND on 08/10/20. Re-use and distribution is strictly not permitted, except for Open Access articles.

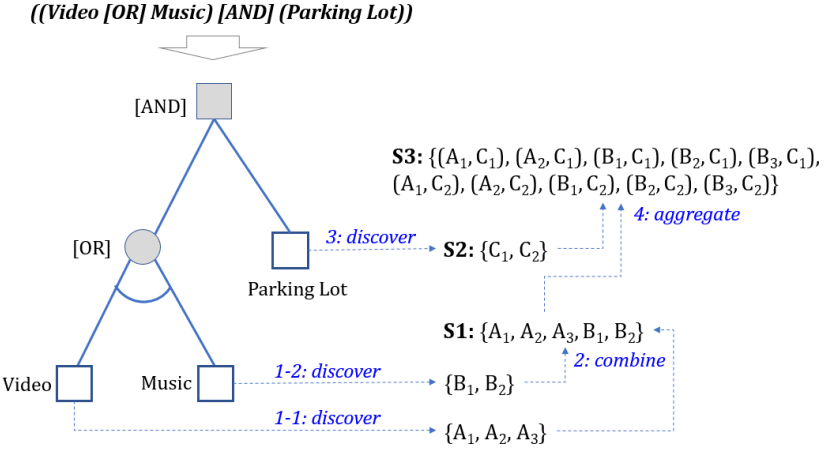


Fig. 6. Example of complex SOT using logical operators.

Music RTs (step 2 in Fig. 6) to produce a new set with five services (referred to here as set S_1). SFT also processes "[AND] (Parking Lot)" to allocate a thread for the "Parking Lot" task (step 3 in Fig. 6) to find two candidate services (referred to here as set S_2). Finally, SFT determines that there is no subsequent "AND", and therefore begins assembling service combinations through the pairing of sets A and B (step 4 in Fig. 6). The resulting 10 combinations (set S_3 in Fig. 6) are then enclosed within 10 DSBs.

The aim of the service discovery chain is to assess interface compatibility by checking whether the inputs in a QD cover the inputs of the current candidate service. If the available inputs are incapable of invoking the candidate service, then the supported service handler and aided service hander search for supplementary services to generate the additional data required to invoke the candidate service. Figure 7 illustrates the process of the service discovery chain. The four handlers in the service discovery chain in Fig. 7 help users to find suitable services. Note that each handler has a different task:

(1) Pre-Processing Handler: It prepares input parameters in QD and stores them within an IP.

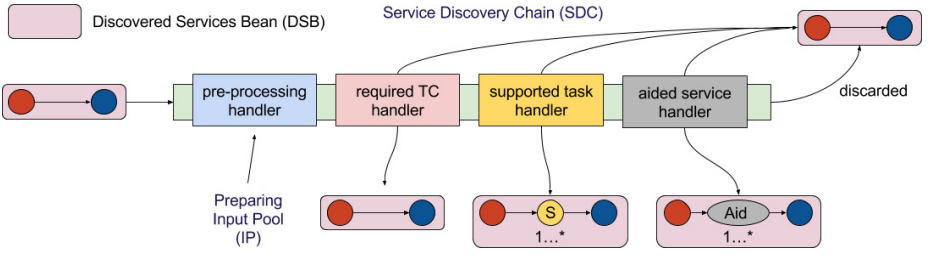


Fig. 7. Process of SDC.

(2) Required Task Handler: It determines whether the input parameters of IP cover the input parameters of the candidate service by calculating the AHMS, collecting the skipped input parameters, and transforming the qualified DSBs into an LOBC.
(3) Supported Service Handler: It finds suitable supported services to provide the skipped input parameters required for the user-given supported tasks in the QD, recalculates the AHMS of candidate services by incorporating the identified supported service, and transforms the qualified DSBs into an LOBC. Note that this handler also calculates HMS to discard unsuitable supported services with inputs that are not provided by the IP.
(4) Aided Service Handler: If SSH is unable to find services to provide skipped parameters, then the ASH takes over the task by scanning the entire graph database for such services. The major difference between supported service and aided service is that the retrieval of supported services is based on the user-given supported tasks, whereas the retrieval of aided services is based on an exhaustive search of the graph DB. Note that if the SSH and ASH both prove unable to provide the necessary input parameters, then TAD discards the DSB.

Back to the example of service composition in Fig. 5. A large number of DSBs are discarded during the composition and discovery process, such that only qualified DSBs are sent to the LOBC to find the next service(s) for the SOT. When it receives a DSB, the LOBC checks whether the last service of the DSB serves the final RT in the AOT (as shown in Fig. 8). When the service for the final RT is ready, the LOBC calculates an RS for the candidate service compositions and retains the compositions. Note that RS considers semantic similarity as well as interface compatibility using Eq. (4). The compositions are stored in the result list until the other composition processes are completed. TAD ranks all candidate service compositions in the composition list according to their RSs, and returns the list to the user to issue a QD.

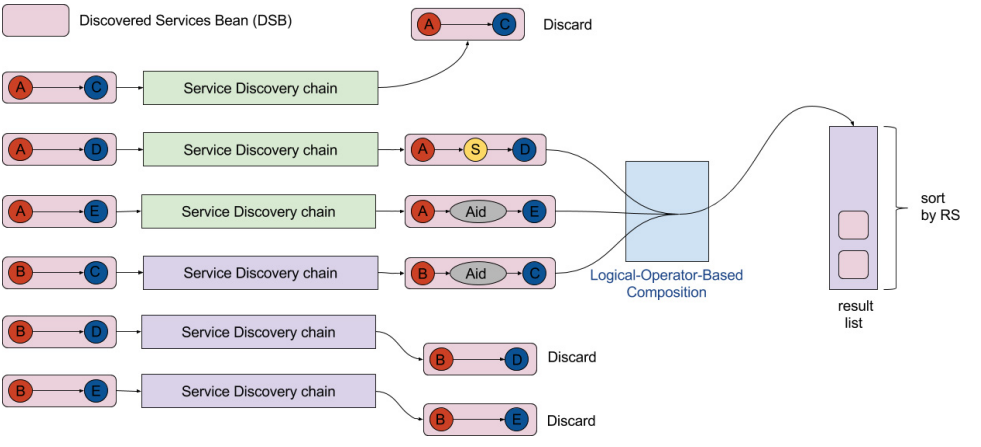


Fig. 8. Final steps (closure phase) of the proposed composition process.

Int. J. Soft. Eng. Knowl. Eng. 2020.30:835-857. Downloaded from www.worldscientific.com
by UNIVERSITY OF NEW ENGLAND on 08/10/20. Re-use and distribution is strictly not permitted, except for Open Access articles.

Int. J. Soft. Eng. Knowl. Eng. 2020.30:835-857. Downloaded from www.worldscientific.com
by UNIVERSITY OF NEW ENGLAND on 08/10/20. Re-use and distribution is strictly not permitted, except for Open Access articles.

4. System Prototype

To demonstrate the feasibility of the proposed approach, we have built a system prototype in the form of a Web application. The main functionality of the TAD system is generating a QD by inputting a Web form and providing recommended service compositions based on the given QD. Besides, the TAD system also allows the users to publish OAS documents in JSON format. The TAD system is developed by using *Spring Boot* for the backend APIs, *React* for the front-end user interfaces, and *Neo4J* for the data storage. For the user interface of providing input and output parameters, the user can fill in the concept words for parameters in the input field, separating them with commas (the left part of Fig. 9). When filling in the SOT, the user can use * to indicate ST in addition to the normal RT. Notably, the TAD system parses the SOT immediately, with RTs shown in blue, STs in yellow, and logic operators in red. The user then can press SEARCH to begin looking for service compositions.

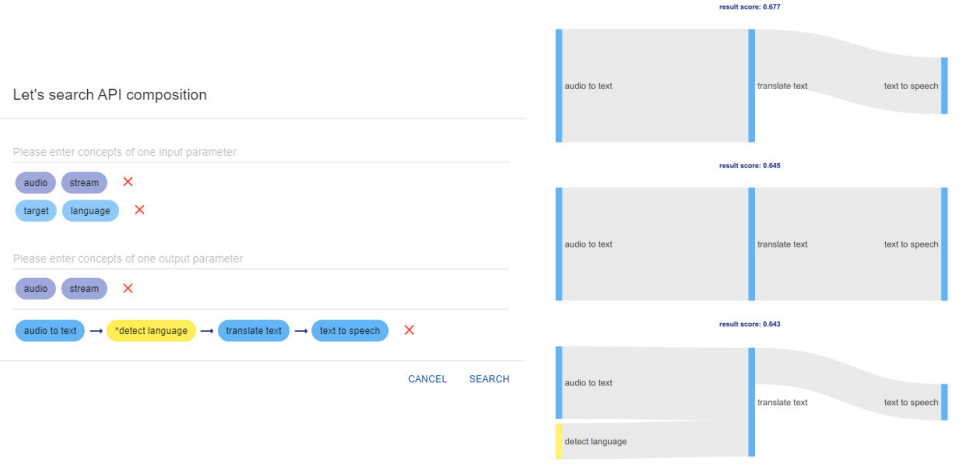


Fig. 9. GUI for filling out SOT and the search results.

The search results are presented using multiple Sankey Diagrams (the right part in Fig. 9), each of which represents a candidate service composition with an RS. The bar in the graph represents a service. The service compositions are ranked according to their RSs. The thickness of the bar represents the number of parameters. The user can click the bar to see details of a candidate service, including the service type, service task, service title, service provider, service interface path, and the calculated scores CS and AHMS. Clicking the zone between two bars (i.e. the preceding service and the succeeding one) allows the user to browse the mapping from the output parameters of the preceding service or the data provided by the user pertaining to the input parameters of the succeeding service (Fig. 10).



Fig. 10. GUI of details for services and linkages.

5. Experimental Evaluations

The feasibility of the TAD prototype system was evaluated by composing 10 query documents. In particular, we are interested in answering the following two research questions (RQs).

- RQ1: Is the proposed TAD approach helpful for the retrieval of RESTful service compositions? We would like to study whether the service compositions recommended by TAD were reasonable and satisfactory to the user.
- RQ2: Can the proposed TAD approach produce service compositions efficiently? We would like to study whether the whole service composition process could be executed in an acceptable time period.

In the following, we first introduce the precision indicator used in the experiments. We then describe the test data. Finally, we analyze the experimental results and discuss a number of overriding issues.

5.1. *Evaluation indicators*

The Top-K precision indicator refers to the fraction of retrieved Top-K data items that are relevant to the user query. This metric is commonly used to evaluate the accuracy of data search systems; however, it is insufficient when seeking to verify a service composition system. It is difficult to classify a service composition as entirely relevant or entirely irrelevant to the requirements stipulated by the user. Thus, we devised a new composition precision (CP) indicator, as follows:

$$\mathrm{CP}^k(\mathrm{QD}) = \frac{|\mathrm{SRT}^k_{\mathrm{relevant}}|}{|\mathrm{SRT}^k|}, \tag{5}$$

where QD is the query document, SRT^k denotes the services discovered for RTs in the kth returned candidate composition, and $\mathrm{SRT}_{\mathrm{relevant}}$ is the relevant services for RTs. Composition precision refers to the fraction of relevant RT services over the number of all RT services.

Int. J. Soft. Eng. Knowl. Eng. 2020.30:835-857. Downloaded from www.worldscientific.com
by UNIVERSITY OF NEW ENGLAND on 08/10/20. Re-use and distribution is strictly not permitted, except for Open Access articles.

Int. J. Soft. Eng. Knowl. Eng. 2020.30:835-857. Downloaded from www.worldscientific.com
by UNIVERSITY OF NEW ENGLAND on 08/10/20. Re-use and distribution is strictly not permitted, except for Open Access articles.

Note that we consider only the services identified for RTs; that is, supported and aided services are merely supplemental to the composition. The relevant services are determined by two guidelines: (1) The service meets the semantic concept of the corresponding RT; and (2) the user parameters, the outputs of pre-chained services, and supplemental services cover most of the inputs required for successful invocation of the service.

Another measure, response time, can also be used to illustrate the efficiency by calculating the time (in seconds) from the moment QD is received to the moment at which the composition list is returned.

5.2. *Comparison target: Semi-automatic composition approach*

To verify the efficacy of the proposed service composition scheme, we also implemented semi-automated composition (SAC), the details of which are outlined in the following:

(1) The user manually specifies the input and output parameters of each component service in the desired service composition. For example, an SOT [“*audio to text*”, “*translate text*”, “*text to speech*”] should be split into three SOTs for each RT: [“*audio to text*”],[“*translate text*”], and [“*text to speech*”].
(2) Candidate component services are retrieved using the proposed TAD, wherein the system is run several times to retrieve candidate component services for each SOT with only one required task.
(3) The candidate services of relevant SOTs are combined to produce composite services, which are then ranked according to the sum of their RSs. For example, if there are three candidate services for [“*audio to text*”], two services for [“*translate text*”], and four services for [“*text to speech*”], then a total of 24 composite services are produced for ranking based on the summed RSs.

5.3. *OAS documents of real-world RESTful services and query documents*

The source of the RESTful services in this study was the open-source projects of OAS (APIs.guru) [5]. We also wrote a number of OAS documents for real-world Web APIs to enlarge the dataset. After mapping all of the OAS documents to a graph in Neo4J, there were 8629 endpoints on which to base the experiments. Ten representative query documents in a variety of domains were prepared for the experiments (see Table 2).

We also developed a set of query documents to evaluate the precision of the SAC approach (see Table 3). Basically, these test cases split the complex SOTs in Table 2 into several simple SOTs that include only one RT. Note that Test Cases #1 and #2 are not included because they were originally simple SOTs. We also opted not to include Test Case #10 due to the fact that breaking down such a complex SOT would be unreasonable.

Int. J. Soft. Eng. Knowl. Eng. 2020.30:835-857. Downloaded from www.worldscientific.com
by UNIVERSITY OF NEW ENGLAND on 08/10/20. Re-use and distribution is strictly not permitted, except for Open Access articles.

Table 2. Query documents.

Test case	Input parameters	Output parameters	Sequence of tasks
Test Case #1	[“latitude”, “coordinate”], [“longitude”, “coordinate”], [“place”, “address”]	[“weather”]	[“find worldwide weather by location”]
Test Case #2	[“target”, “language”], [“text”, “translate”]	[“translate”, “text”, “result”]	[“translate text”]
Test Case #3	[“target”, “language”], [“text”, “translate”]	[“translate”, “text”, “result”]	[“*detect text language”, “translate text”]
Test Case #4	[“target”, “language”], [“audio”, “stream”]	[“audio”, “stream”]	[“audio to text”, “translate text”, “text to speech”]
Test Case #5	[“image”, “picture”], [“sentences”, “length”]	[“text”, “summary”, “sentences”]	[“recognize image to tag”, “provide article or blog”, “text to summary”]
Test Case #6	[“place”, “address”, “city”]	[“weather”, “location”]	[“convert place to geographic location”, “find weather by geographic location”]
Test Case #7	[“radius”, “scope”], [“coordinate”, “latitude”], [“coordinate”, “longitude”], [“place”, “location”, “address”]	[“image”, “picture”]	[“search event by coordinate or place”, “search images”]
Test Case #8	[“image”, “picture”], [“sentences”, “length”]	[“text”, “summary”, “sentences”]	[“recognize image to tag”, “(search article [OR] search news)”, “text to summary”]
Test Case #9	[“place”, “address”, “city”, “query”]	[“weather”, “location”], [“video”, “stream”, “picture”, “image”]	[“*place to geographic coordinates”, “(search youtube video [AND] search weather by place or location)”]
Test Case #10	[“place”, “address”, “location”], [“coordinate”, “latitude”], [“coordinate”, “longitude”]	[“store”, “food”, “mall”, “restaurant”], [“parking lot”, “event”]	[“((search food restaurant [OR] search shopping mall) [AND] (search parking lot [OR] search event))”]

5.4. *Experiment results*

The 10 query documents in Table 2 were issued to the TAD prototype, and the 18 query documents in Table 3 were issued to the SAC implementation. We then calculated the Top-1, Top-2, and Top-3 CPs, and recorded the average response time of each query (executed 10 times). The execution environment for the TAD prototype in our experiments was a MacBook Air with a 1.6-GHz Intel Core i5 with 8-GB RAM and 250-GB hard disk running MacOS Sierra 10.12.4 (64-bit).

Table 4 presents the experimental results obtained by applying TAD to the test cases listed in Table 2. Those results show that TAD yielded good composition precision (0.75–1.0) in most of the cases, which should result in service compositions capable of satisfying user requirements. Note that TAD was able to generate at least

Int. J. Soft. Eng. Knowl. Eng. 2020.30:835-857. Downloaded from www.worldscientific.com
by UNIVERSITY OF NEW ENGLAND on 08/10/20. Re-use and distribution is strictly not permitted, except for Open Access articles.

Table 3. Query documents for the semi-automatic approach.

Test case	Input parameters	Output parameters	Sequence of tasks
Test Case #3-1	[“target”, “language”], [“text”, “translate”]	[“text”, “language”]	[“detect text language”]
Test Case #3-2	[“target”, “language”], [“text”, “translate”]	[“translate”, “text”, “result”]	[“translate text”]
Test Case #4-1	[“audio”, “stream”]	[“text”, “result”]	[“audio to text”]
Test Case #4-2	[“target”, “language”], [“text”, “result”]	[“translate”, “text”, “result”]	[“translate text”]
Test Case #4-3	[“translate”, “text”, “result”]	[“audio”, “stream”]	[“text to speech”]
Test Case #5-1	[“image”, “picture”]	[“text”, “tag”]	[“recognize image to tag”]
Test Case #5-2	[“sentences”, “length”], [“text”, “tag”]	[“text”, “sentences”]	[“provide article or blog”]
Test Case #5-3	[“text”, “sentences”]	[“text”, “summary”, “sentences”]	[“text to summary”]
Test Case #6-1	[“place”, “address”, “city”]	[“coordinate”, “latitude”], [“coordinate”, “longitude”]	[“convert place to geographic location”]
Test Case #6-2	[“coordinate”, “latitude”], [“coordinate”, “longitude”]	[“weather”, “location”]	[“find weather by geographic location”]
Test Case #7-1	[“radius”, “scope”], [“coordinate”, “latitude”], [“coordinate”, “longitude”], [“place”, “location”, “address”]	[“event”, “text”]	[“search event by coordinate or place”]
Test Case #7-2	[“event”, “text”]	[“image”, “picture”]	[“search images”]
Test Case #8-1	[“image”, “picture”]	[“text”, “tag”]	[“recognize image to tag”]
Test Case #8-2	[“sentences”, “length”], [“text”, “tag”]	[“text”]	[“(search article [OR] search news)”]
Test Case #8-3	[“text”, “sentences”]	[“text”, “summary”, “sentences”]	[“text to summary”]
Test Case #9-1	[“place”, “address”, “city”, “query”]	[“coordinate”, “latitude”], [“coordinate”, “longitude”]	[“place to geographic coordinates”]
Test Case #9-2	[“coordinate”, “latitude”], [“coordinate”, “longitude”], [“video”, “audio”]	[“weather”, “location”], [“video”, “stream”, “picture”, “image”]	[“(search youtube video [AND] search weather by place or location)”]

Table 4. Experimental results obtained using TAD.

Test case	Top-1 AP	Top-2 AP	Top-3 AP	Average response time
Test Case #1	1.000	N/A	N/A	1.055
Test Case #2	1.000	1.000	1.000	4.242
Test Case #3	1.000	1.000	1.000	11.219
Test Case #4	1.000	1.000	1.000	37.259
Test Case #5	1.000	1.000	1.000	12.310
Test Case #6	0.500	0.500	0.667	5.536
Test Case #7	1.000	1.000	N/A	16.506
Test Case #8	1.000	1.000	1.000	296.033
Test Case #9	1.000	1.000	0.889	4.224
Test Case #10	1.000	0.750	N/A	35.335

Int. J. Soft. Eng. Knowl. Eng. 2020.30:835-857. Downloaded from www.worldscientific.com
by UNIVERSITY OF NEW ENGLAND on 08/10/20. Re-use and distribution is strictly not permitted, except for Open Access articles.

Table 5. Experiment results obtained using SAC.

Test case	Top-1 AP	Top-2 AP	Top-3 AP	Average response time
Test Case #3	1.000	1.000	0.833	18.192
Test Case #4	0.667	0.667	0.556	89.854
Test Case #5	N/A	N/A	N/A	10.756
Test Case #6	0.500	0.750	N/A	26.441
Test Case #7	1.000	1.000	N/A	11.381
Test Case #8	N/A	N/A	N/A	14.596
Test Case #9	1.000	N/A	N/A	11.472

one perfect service composition for each of the various queries. The response times of the test cases varied between 3 s and 3 min. Taking the Test Case #3 as an example, "article" and "text" are common keywords for a great number of services; therefore, the LOBC module retrieved a large number of services that had to be checked for interface compatibility. Nevertheless, TAD was still able to produce feasible service compositions for this query. Note also that the time required to complete this action was far shorter than would be required if discovery and composition have to be performed manually.

Table 5 lists the experimental results obtained when SAC was applied to the test cases in Table 3. Note that the AP results are already aggregated in accordance with the methods in Sec. 5.2. The response time here is the sum of the response times involving the retrieval of component services. For example, the response time for Test Case #3 is the sum of the response times for Test Case #3-1 and Test Case #3-2. No suitable composite services were identified for many of the test cases, due to the fact that the specifications for the input and output parameters were too strict to find component services.

The performances of precision and response time of both TAD and SAC are shown in Fig. 11. Based on the performance results, the research questions are answered as follows.

- RQ1: The precision of the produced service compositions.

As shown in Figs. 11(a)–11(c), TAD clearly outperformed SAC from the viewpoint of precision, especially for the fact that TAD was able to identify service compositions for all of the cases. Although SAC outperformed TAD in one special case (AP-2 in Test Case #6), this can be explained by the fact that specifying the bridging parameters of latitude and longitude can be helpful in locating component services suitable for the expected service flow. In general, TAD can produce reasonable and satisfactory service compositions.

- RQ2: The efficiency of the service composition process.

The response times of TAD varied between 3 s and 3 min with an average of about 40 s; it is acceptable for an assistant tool for software developers. Note that the response time of TAD when applied to Test Case #8 far exceeded the response time of SAC [see Fig. 11(d)]; however, TAD succeeded in finding a service composition that satisfied the query document, whereas SAC failed to do so. Briefly, TAD is able to

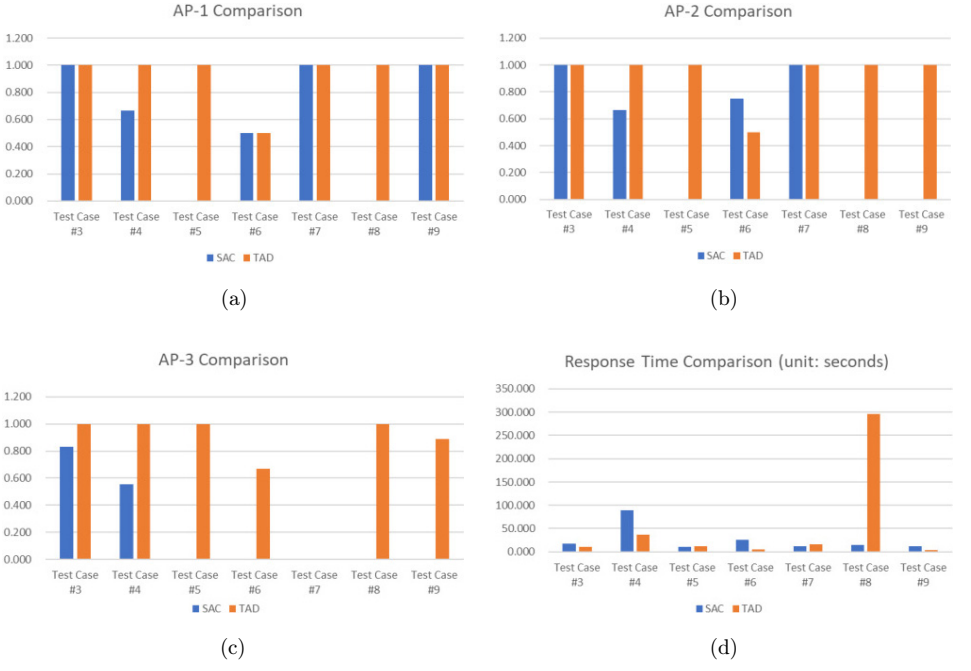


Fig. 11. Comparisons of SAC and TAD with respect to (a) AP-1, (b) AP-2, (c) AP-3, and (d) response times.

provide service compositions that satisfy user requirements in an acceptable time period.

6. Conclusions

This paper presents a novel approach to the discovery and composition of real-world RESTful services. The proposed scheme, TAD, features four main attributes: (1) TAD transforms OAS documents into graph nodes on which semantic annotation is performed to facilitate service retrieval; (2) a newly-devised QD that includes concepts of inputs, concepts of outputs, and the SOT, helps users to stipulate composition requirements; (3) a service discovery chain checks service interface compatibility for the retrieval of supported and aided services in order to ensure the feasibility of compositions; and (4) TAD supports a variety of service flow structures, including branch and parallel patterns as well as sequential patterns. Experiment results demonstrate the feasibility and search precision of the TAD approach.

Future developments will focus on enhancing the semantic annotation mechanism through the use of an ontology database and semantic lifting/lowering mechanisms to enable the retrieval of more effective concepts and to shorten the response times. Besides, another plan is devising a method to map or decompose a source parameter

Int. J. Soft. Eng. Knowl. Eng. 2020.30:835-857. Downloaded from www.worldscientific.com
by UNIVERSITY OF NEW ENGLAND on 08/10/20. Re-use and distribution is strictly not permitted, except for Open Access articles.

into multiple target parameters to further enhance the possibility of service composition.

Int. J. Soft. Eng. Knowl. Eng. 2020.30:835-857. Downloaded from www.worldscientific.com
by UNIVERSITY OF NEW ENGLAND on 08/10/20. Re-use and distribution is strictly not permitted, except for Open Access articles.

Acknowledgments

This research was sponsored by the Ministry of Science and Technology in Taiwan under the Grants MOST 105-2221-E-019-054-MY3 and 108-2221-E-019-026-MY3.

References

1. R. T. Fielding and R. N. Taylor, Principled design of the modern Web architecture, *ACM Trans. Internet Technol.* **2** (2002) 115–150.
2. ProgrammableWeb, LLC, ProgrammableWeb, 2019, http://www.programmableweb.com/.
3. scale, APIEvangelist and APItools.com, APIs.io, 2019, http://apis.io/.
4. Kong, Mashape, 2019, https://www.mashape.com/.
5. APIs.guru, APIs.guru, 2019, https://apis.guru/.
6. M. Garriga, C. Mateos, A. Flores, A. Cechich and A. Zunino, RESTful service composition at a glance: A survey, *J. Netw. Comput. Appl.* **60** (2016) 32–53.
7. R. Alarcon, R. Saffie, N. Bravo and J. Cabello, REST Web service description for graph-based service discovery, in *ICWE 2015: Engineering the Web in the Big Data Era*, eds. P. Cimiano, F. Frasincar, G.-J. Houben and D. Schwabe, Lecture Notes in Computer Science, Vol. 9114 (Springer, Cham, 2015), pp. 461–478.
8. C. Pautasso and G. Alonso, The JOpera visual composition language, *J. Vis. Lang. Comput.* **16**(1–2) (2005) 119–152.
9. R. Alarcon, E. Wilde and J. Bellido, Hypermedia-driven RESTful service composition, in *ICSOC 2010: Service-Oriented Computing*, Lecture Notes in Computer Science, Vol. 6568 (Springer, Berlin, 2011), pp. 111–120.
10. L. Zeng, B. Benatallah, M. Dumas, J. Kalagnanam and Q. Z. Sheng, Quality driven Web services composition, in *Proc. 12th Int. Conf. World Wide Web* (ACM, New York, 2003), pp. 411–421.
11. L. Zeng, B. Benatallah, A. H. H. Ngu, M. Dumas, J. Kalagnanam and H. Chang, QoS-aware middleware for Web services composition, *IEEE Trans. Softw. Eng.* **30**(5) (2004) 311–327.
12. R. Aggarwal, V. Kunal, J. Miller and W. Milnor, Constraint driven Web service composition in METEOR-S, in *Proc. IEEE Int. Conf. Services Computing*, 2004, pp. 23–30.
13. T. Yu and K.-J. Lin, Service selection algorithms for Web services with end-to-end QoS constraints, *Inf. Syst. e-Bus. Manage.* **3**(2) (2005) 103–126.
14. L. Qi, Y. Tang, W. Dou and J. Chen, Combining local optimization and enumeration for QoS-aware Web service composition, in *Proc. IEEE Int. Conf. Web Services* (IEEE Computer Society, 2010), pp. 34–41.
15. M. Alrifai and T. Risse, Combining global optimization with local selection for efficient QoS-aware service composition, in *Proc. 18th Int. Conf. World Wide Web* (ACM, New York, 2009), pp. 881–890.
16. J. Lee, S.-P. Ma, Y.-Y. Lin, S.-J. Lee and Y.-C. Wang, Dynamic service composition: A discovery-based approach, *Int. J. Softw. Eng. Knowl. Eng.* **18**(2) (2008) 199–222.

17. SmartBear Software, OpenAPI Specification (OAS), 2020, https://swagger.io/docs/specification/.
18. D. M. Blei, A. Y. Ng and M. I. Jordan, Latent Dirichlet allocation, *J. Mach. Learn. Res.* **3** (2003) 993–1022.
19. G. A. Miller, WordNet: a lexical database for English, *Commun. ACM* **38**(11) (1995) 39–41.
20. M. Alrifai, T. Risse and W. Nejdl, A hybrid approach for efficient Web service composition with end-to-end QoS constraints, *ACM Trans. Web* **6**(2) (2012) 7:1–7:31.
21. D. Grigori, J. C. Corrales and M. Bouzeghoub, Behavioral matchmaking for service retrieval, in *Proc. IEEE Int. Conf. Web Services*, 2006, pp. 145–152.
22. A. Cuzzocrea and M. Fisichella, Discovering semantic Web services via advanced graph-based matching, in *Proc. IEEE Int. Conf. Systems, Man, and Cybernetics*, 2011, pp. 608–615.
23. H. Ed-douibi, J. L. Cánovas Izquierdo and J. Cabot, APIComposer: Data-driven composition of REST APIs, in *Service-Oriented and Cloud Computing*, Lecture Notes in Computer Science, Vol. 11116 (Springer, Cham, 2018), pp. 161–169.
24. J. Webber, A programmatic introduction to Neo4j, in *Proc. 3rd Annu. Conf. Systems, Programming, and Applications: Software for Humanity*, 2012, pp. 217–218.
25. S.-P. Ma, Y.-Y. FanJiang and J.-Y. Kuo, Dynamic service composition using core service identification, *J. Inf. Sci. Eng.* **30**(4) (2014) 957–972.
26. S.-P. Ma, H.-J. Lin, C.-W. Lan, W.-T. Lee and M.-J. Hsu, Real-world RESTful service composition: A transformation-annotation-discovery approach, in *Proc. IEEE 10th Conf. Service-Oriented Computing and Applications*, 2017, pp. 1–8.
27. Y. Li, Z. A. Bandar and D. McLean, An approach for measuring semantic similarity between words using multiple information sources, *IEEE Trans. Knowl. Data Eng.* **15**(4) (2003) 871–882.
28. S.-P. Ma, H.-J. Lin, H.-M. Chen, Y.-J. Chen and W.-T. Lee, Web API discovery using semantic similarity and hungarian algorithm, *J. Internet Technol.* **19**(6) (2018) 1657–1664.
29. V. Kumar and L. N. Kanal, Parallel branch-and-bound formulations for AND/OR tree search, *IEEE Trans. Pattern Anal. Mach. Intell.* **PAMI-6**(6) (1984) 768–778.

Int. J. Soft. Eng. Knowl. Eng. 2020.30:835-857. Downloaded from www.worldscientific.com
by UNIVERSITY OF NEW ENGLAND on 08/10/20. Re-use and distribution is strictly not permitted, except for Open Access articles.